

```
In [35]: # ✅ 创建并写入（文件已存在则清空覆盖）
with open('data.txt', 'w', encoding='utf-8') as f:
    f.write('Hello, World!')

# ✅ 创建并追加（文件已存在则保留原内容，在末尾追加）
with open('data.txt', 'a', encoding='utf-8') as f:
    f.write('\n')
    f.write('New line')
```

```
In [30]: # 覆盖写入
with open('output.txt', 'w', encoding='utf-8') as f:
    f.write('Hello, World!\n')      # 写入字符串
    f.writelines(['Line1\n', 'Line2\n']) # 写入字符串列表

# 追加写入
with open('log.txt', 'a', encoding='utf-8') as f:
    f.write('New log entry\n')
```

```
In [34]: with open('data.txt', 'r', encoding='utf-8') as f:
    f.seek(0)      # 移动到文件开头
    f.seek(10)     # 移动到第10个字节位置
    pos = f.tell() # 获取当前指针位置
```

```
In [36]: def fun1():
    with open('data.txt', 'a', encoding='utf-8') as file:
        file.write('\n')
        file.write('zhangsan')
        file.write('\n')
        lst = [1,2,3]
        file.write(str(lst))

fun1()
```

```
In [43]: import os

USER_FILE = 'user_data.txt'

def file_exists():
    """确保用户数据文件存在，避免读取时报错"""
    if not os.path.exists(USER_FILE):
        with open(USER_FILE, 'w', encoding='utf-8') as f:
            pass # 创建空文件

def regeister_user():
    print("=== 用户注册 ===")
    username = input("请输入用户名:")
    password = input("请输入密码:")

    if not username or not password:
        print("❌ 用户名和密码不能为空！")
        return

    file_exists() # 确保文件存在
    with open(USER_FILE, 'r', encoding='utf-8') as f:
        for line in f:
            stored_user = line.strip().split('|')[0]
            if stored_user == username:
                print(f"❌ 用户名 '{username}' 已被注册，请更换！")
                return
```

```

with open(USER_FILE, 'a', encoding='utf-8') as f:
    f.write(f"{username}|{password}\n")

print(f"用户 '{username}' 注册成功")

def login_user():
    print("=== 用户登录 ===")
    username = input("请输入用户名:")
    password = input("请输入密码:")

    if not username or not password:
        print("❌ 用户名和密码不能为空!")
        return

    is_success = False
    file_exists() # 确保文件存在
    with open(USER_FILE, 'r', encoding='utf-8') as f:
        for line in f:
            stored_user, stored_password = line.strip().split('|')
            if stored_user == username and stored_password == password:
                is_success = True
                break
            print(f"✅ 用户 '{username}' 登录成功!")

    print("❌ 用户名或密码错误!")
    if is_success:
        print("登录成功")
    else:
        print("登录失败, 用户名或密码错误!")

def main():
    """主程序入口：菜单驱动循环"""
    while True:
        print("\n" + "="*30)
        print("      用户管理系统")
        print("="*30)
        print("1. 注册")
        print("2. 登录")
        print("0. 退出")
        print("-"*30)

        choice = input("请选择操作 (0/1/2): ").strip()

        if choice == '1':
            register_user()
        elif choice == '2':
            login_user()
        elif choice == '0':
            print("👋 感谢使用，再见!")
            break
        else:
            print("⚠️ 无效选择，请输入 0、1 或 2")

if __name__ == "__main__":
    main()

```

```

=====
                用户管理系统
=====
1. 注册
2. 登录
0. 退出
-----
=== 用户注册 ===
用户'lihua'注册成功

=====
                用户管理系统
=====
1. 注册
2. 登录
0. 退出
-----
=== 用户注册 ===
❌ 用户名 'lisi' 已被注册，请更换！

=====
                用户管理系统
=====
1. 注册
2. 登录
0. 退出
-----
👋 感谢使用，再见！

```

```

In [44]: def copy_user_data():
# 4.3 需求 & 4.4 分析：明确源文件和目标文件
src_file = "user_data.txt"
dst_file = "user_data-2.txt"

try:
# 4.4 分析步骤1 & 4.2 步骤1：打开别人的文件(源)，自己的文件(目标)
with open(src_file, 'r', encoding='utf-8') as src, \
    open(dst_file, 'w', encoding='utf-8') as dst:

# 4.4 分析步骤2 & 4.2 步骤2：读取别人文件中的数据，写到自己的文件中
for line in src:
    dst.write(line)

    print(f"✅ 复制成功！已生成新文件：{dst_file}")

except FileNotFoundError:
    print(f"❌ 错误：找不到源文件 '{src_file}'，请确认该文件是否存在。")
except Exception as e:
    print(f"❌ 复制失败：{e}")

if __name__ == '__main__':
    copy_user_data()

```

✅ 复制成功！已生成新文件：user_data-2.txt

文本文件复制的逻辑与实现

- 核心逻辑说明：文本文件复制的逻辑类比抄作业，需要同时打开原文件和目标文件，读取原文件内容后写入目标文件，最后关闭两个文件。

- 代码实现步骤：第一步用 `r` 模式、`utf-8` 编码打开原文件 `user.txt`，调用 `readlines` 读取全部内容后关闭原文件；第二步用 `w` 模式、`utf-8` 编码打开目标备份文件 `users_bak.txt`，调用 `writelines` 写入内容后关闭目标文件。
- 执行结果验证：调用函数执行后，生成的 `users_bak.txt` 内容和原文件完全一致，实现了复制效果。

非文本文件（图片/视频）的复制实现

- 常见错误分析：直接沿用文本文件的读写方法复制图片会报错，因为图片以二进制存储，不适用 `utf-8` 文本编码。
- 二进制模式说明：在Python文件打开模式中，添加 `b` 后缀即可切换为二进制模式，`rb` 代表二进制只读，`wb` 代表二进制只写，均适用于图片、视频等非文本文件操作。
- 二进制复制步骤：用 `rb` 模式打开原图片，调用 `read` 一次性读取全部内容，再用 `wb` 模式打开目标图片，调用 `write` 一次性写入内容即可。
- 执行结果验证：修改模式后执行代码，生成的备份图片和原图片尺寸、内容完全一致，复制成功。

OS模块的基础定位与导入方法

- 模块功能说明：OS模块提供和操作系统交互的功能，主要用于操作文件夹和路径，文本文件的读写仍然使用基础的 `open` 函数操作。
- 核心学习目标：本次课程侧重讲解四个常用功能：获取指定路径下所有文件名、拼接路径、判断路径类型、创建不存在的目录。
- 导入方式：使用OS模块前，需要通过 `import os` 语句导入模块。

使用os.listdir获取指定目录下的文件名

- 函数功能：`os.listdir` 接收一个路径参数，返回该路径下第一层所有文件和文件夹的名称，返回结果为列表格式。
- 路径书写规则：Windows系统的绝对路径需要在字符串前加 `r`，关闭转义符的转义效果，避免路径解析错误。
- 使用场景：当需要批量处理指定文件夹下的所有文件时，需要先通过该函数获取所有文件的名称，再做后续处理。

使用os.path.join拼接路径

- 核心优势：相比手动字符串拼接，`os.path.join` 不需要手动添加路径分隔斜杠，会自动根据操作系统适配分隔符。
- 参数规则：`os.path.join` 支持传入不定长数量的参数，可以一次性拼接多级目录和文件名，得到完整的最终路径。
- 路径类型规则：如果传入的基础路径是绝对路径，拼接后得到的就是绝对路径；如果传入的基础路径是相对路径，拼接后得到的就是相对路径。

```
In [45]: def copy_binary_file(src_name, dst_name):
        try:
            with open(src_name, 'rb') as src, open(dst_name, 'wb') as dst:
                # 一次性读取全部内容并写入（小文件推荐）
                data = src.read()
                dst.write(data)
```

```

        print(f"✅ 复制成功: {src_name} → {dst_name}")
    except FileNotFoundError:
        print(f"❌ 文件不存在: {src_name}")
    except Exception as e:
        print(f"❌ 复制失败: {e}")

# 使用示例
if __name__ == '__main__':
    copy_binary_file("01.jpg", "01-副本.jpg")

```

✅ 复制成功: 01.jpg → 01-副本.jpg

```

In [48]: def fun3():
        path = r"D:\project\step1\week3"
        files_name = os.listdir(path)
        print(files_name)

        if __name__ == '__main__':
            fun3()

```

```

['01-副本.jpg', '01.jpg', '0601.ipynb', '0601.py', 'data', 'data.txt', 'log.txt',
'output.txt', 'user_data-2.txt', 'user_data.txt']

```

```

In [60]: import json

# 写入
data = {"city": "北京", "population": 21540000}
with open("city.json", "w", encoding="utf-8") as f:
    json.dump(data, f, ensure_ascii=False, indent=4)

# 读取验证
with open("city.json", "r", encoding="utf-8") as f:
    loaded = json.load(f)

print(loaded)          # {'city': '北京', 'population': 21540000}
print(type(loaded))    # <class 'dict'>

```

```

{'city': '北京', 'population': 21540000}
<class 'dict'>

```

常见Python文件类型分类与用途

- 标注文件：JSON和XML：计算机视觉方向做目标检测时，目标框坐标标注不会直接修改原图片，会保存在独立的JSON或XML标注文件中。常规目标检测只需要标注矩形框，用左上角(x1,y1)和右下角(x2,y2)即可确定框位置，语义分割标注更复杂，医学图像和缺陷检测场景使用较多。
- 配置文件：YAML：YAML文件通常用作项目工程的配置文件，最常用的场景是数据集配置和神经网络模型配置，后续遇到对应场景会再详细讲解。

JSON格式语法规则讲解

- 基础语法规则：JSON格式和Python字典结构非常相似，用大括号表示单个对象，对象内部以 `key:value` 键值对存储数据，多个数据之间用逗号分隔；多个对象需要用中括号包裹。
- 嵌套规则说明：JSON支持嵌套结构，中括号可以作为value出现在任意层级，不一定只能放在最外侧，比如描述人左右双眼的信息，可以在眼睛key对应的value中用中括号存储两个眼睛对象。

Python字典转JSON存储实操

- with语句打开文件的优势：使用 `with open() as file` 语法打开文件，不需要手动调用 `file.close()` 关闭文件，代码执行完with块后会自动关闭文件，写法更简洁安全。
- 编码转换步骤：要把Python字典写入文件，不能直接写入字典对象，因为write方法仅支持字符串输入。需要先导入json模块，调用 `json.dumps()` 方法将字典对象转换为JSON格式的字符串，再写入文件。
- 格式注意事项：JSON格式要求字符串必须使用双引号，存储数据时尽量统一使用双引号，避免解析出错；如果保存为.json后缀文件，必须严格遵守JSON格式要求，多个对象必须用中括号包裹，否则会出现格式解析错误。
- 实际应用场景说明：实际开发中很少需要自己从字典生成JSON标注文件，JSON标注文件一般是标注工具生成的，开发中更多是做JSON文件读取解析工作。

```
In [67]: #作业1
import os

def print_all_dirs(base_dir):
    """
    打印 base_dir 下的所有子目录（仅第一层）
    """
    if not os.path.isdir(base_dir):
        print(f"❌ 路径不存在: {base_dir}")
        return

    print(f"📁 [{base_dir}] 下的所有目录: ")
    count = 0
    for item in sorted(os.listdir(base_dir)): # 排序输出更清晰
        full_path = os.path.join(base_dir, item)
        if os.path.isdir(full_path):
            print(f"📁 {item}")
            count += 1

    print(f"\n✅ 共找到 {count} 个子目录")

base_dir = r"D:\project\step1\week3\MNIST_IMG_TOP9"
print_all_dirs(base_dir)
```

📁 [D:\project\step1\week3\MNIST_IMG_TOP9] 下的所有目录:

📁 test
📁 train

✅ 共找到 2 个子目录

```
In [ ]: #作业2
import csv

data = [
    ['path', 'x', 'y', 'w', 'h'],
    ['1.png', '100', '100', '200', '200'],
    ['2.png', '50', '100', '100', '100'],
    ['3.png', '200', '50', '150', '100'],
    ['4.png', '150', '100', '100', '100']
]

# 写入 CSV 文件（推荐用 csv 模块，安全处理引号/换行）
```

```

with open('annotations.csv', 'w', newline='', encoding='utf-8') as f:
    writer = csv.writer(f)
    writer.writerows(data)

print("✅ 已写入 annotations.csv")

dataDic = {}

with open('annotations.csv', 'r', encoding='utf-8') as f:
    reader = csv.reader(f)
    header = next(reader) # 跳过第一行 (列名)

    for row in reader:
        if len(row) < 5: # 防止空行或格式错误
            continue
        filename = row[0] # '1.png'
        # 将 x, y, w, h 转为整数, 并存入字典
        values = [int(x) for x in row[1:5]] # ['100', '100', ...] → [100, 100, ..]
        dataDic[filename] = values

print("📊 dataDic =", dataDic)

```

✅ 已写入 annotations.csv

📊 dataDic = {'1.png': [100, 100, 200, 200], '2.png': [50, 100, 100, 100], '3.png': [200, 50, 150, 100], '4.png': [150, 100, 100, 100]}

In []: #作业3

```

import os

def load_mnist_image_paths(base_dir):

    def get_all_images(folder_path):
        """遍历 folder_path 下 0~9 子目录, 收集所有 .jpg/.png 文件"""
        paths = []
        for class_dir in os.listdir(folder_path):
            class_path = os.path.join(folder_path, class_dir)
            if not os.path.isdir(class_path):
                continue
            # 只处理 0~9 的目录 (防误匹配其他文件夹)
            if not class_dir.isdigit() or not (0 <= int(class_dir) <= 9):
                continue
            for img_file in os.listdir(class_path):
                if img_file.lower().endswith(('.jpg', '.jpeg', '.png')):
                    full_path = os.path.join(class_path, img_file)
                    paths.append(full_path)
            return sorted(paths) # 字典序排序 (0/1.jpg, 0/10.jpg, 0/100.jpg...)

    trainList = get_all_images(os.path.join(base_dir, "train"))
    testList = get_all_images(os.path.join(base_dir, "test"))

    return trainList, testList

base_dir = r"D:\project\step1\week3\MNIST_IMG_TOP9"

trainList, testList = load_mnist_image_paths(base_dir)

print(f"✅ 训练集图片总数: {len(trainList)}")
print(f"✅ 测试集图片总数: {len(testList)}")

```

```
# 验证: 打印 test/0/ 和 test/9/ 各前2个文件 (确认结构正确)
print("\n 示例路径 (test/0/) :")
for p in testList[:2]:
    if "\\test\\0\\" in p or "/test/0/" in p:
        print(" ", p)

print("\n 示例路径 (train/9/) :")
for p in trainList[-2:]:
    if "\\train\\9\\" in p or "/train/9/" in p:
        print(" ", p)
```

- ✓ 训练集图片总数: 90
- ✓ 测试集图片总数: 90

示例路径 (test/0/) :

D:\project\step1\week3\MNIST_IMG_TOP9\test\0\10.jpg
D:\project\step1\week3\MNIST_IMG_TOP9\test\0\1001.jpg

示例路径 (train/9/) :

D:\project\step1\week3\MNIST_IMG_TOP9\train\9\1005.jpg
D:\project\step1\week3\MNIST_IMG_TOP9\train\9\10055.jpg